TRACES Configuration Guide

Trajectory Research, Analysis, Correction, Exploration, and Simulation — JSON configuration files for CSV import

Table of Contents

- Overview
- Where Configuration Files Live
- Configuration File Structure
- Column Definitions
- Column Definition Items
- Supported Unit Conversions
- Supported Import Functions
- CSV File Requirements
- Data Dictionary
- Columns Created by the Application
- Complete Example
- Adding Import Functions

Overview

TRACES imports vehicle trajectory data from CSV files into a standardized SQLite database for visualization, analysis, and correction. Each CSV file format needs a JSON configuration file that tells the application which columns to expect, how to interpret them, and which database columns to create.

The configuration file is an instruction list for the application: it defines both the structure of the database (which columns exist and their types) and the import rules (how each column is populated from the CSV file). The database schema is entirely configuration-driven — apart from a small set of required columns and the names the application reserves for itself, you are free to define database columns for your own purposes.

Using the Import dialog is covered in the *TRACES User Guide*, section 4.2 "Importing CSV Data"; this guide covers writing the configuration files it uses.

Where Configuration Files Live

Configuration files are kept in a `config` folder in the TRACES user data folder:

- In the installed application, `%APPDATA%\TRACES\config` on Windows.
- When running from source, a `config` folder in the current working directory.

On first launch, if that folder has no `.json` files, TRACES copies its bundled configurations into it (`ohio-one-veh.json`, `ohio-two-veh.json`, `tgsim-L1.json`, `tgsim-L2.json`, `tgsim-moving.json`, `tgsim-stationary.json`). Files already in the folder are never overwritten, so you can edit the copies or add your own.

Each time the Import dialog opens, it reads every `.json` file in the folder and validates it. A file is excluded, with the reason written to the import log, if it:

- isn't valid JSON, or doesn't contain a JSON object at the top level,
- has no `col_defs` key,
- names a `class` that can't be found or isn't an import processor, or
- has any configuration error (described throughout this guide).

The valid files become the file types offered by the Import dialog, each labeled with its filename without the `.json` extension. The configuration is validated again, with every message logged, when an import starts.

Configuration File Structure

A configuration file is a JSON object at the top level. It supports the following top-level keys.

Top-Level Keys

`col_defs` *(required)*

Defines the database columns to create and the rules for populating them from the CSV file. This is the only required top-level key. Its value may be either a JSON object or a JSON array of JSON objects. See Column Definitions for full details.

`file_ext` *(optional)*

The file extension used to filter files in the Import dialog's file chooser. Defaults to `"csv"`.

```
"file_ext": "csv"
```

`class` *(optional)*

The fully qualified Python class name of the processor used to import the file. Defaults to `"data_import.csv_process.CsvProcess"`. The class must be a subclass of `data_import.base_process.BaseProcess`. This key allows alternative import processors (for example, a future JSON file processor) to be selected per configuration.

```
"class": "data_import.csv_process.CsvProcess"
```

Minimal Valid Configuration

```
{
  "col_defs": {
    "event_id":      { "func": "group", "parms": [0, 1], "type": "int" },
    "veh_id":        { "func": "copy",  "parms": [2], "type": "int" },
    "veh_time":      { "func": "copy",  "parms": [3] },
    "lane_id":       { "func": "copy",  "parms": [4], "type": "int" },
    "veh_dist_trav": { "func": "copy",  "parms": [5], "units": "ft" }
  }
}
```

Column Definitions

The `col_defs` value defines one or more sets of database column instructions. Each column definition set is a JSON object whose keys become database column names and whose values are JSON objects containing the column's import instructions.

Column Key Naming Rules

Column keys must follow these rules:

- Lowercase letters only (no digits)
- Optional underscore characters may be used as internal word separators, one at a time
- Must not begin or end with an underscore
- Must not be a name the application creates itself (see Columns Created by the Application)

Examples of valid user-defined keys: `my_column`, `siteid`, `sensor_zone`, `pass_number`

Examples of invalid keys: `MyColumn` (uppercase), `lane_2` (digit), `_private` (leading underscore), `site__id` (double underscore), `imputed_status` (created by the application)

The key name becomes the column name in the SQLite database. Names listed in the Data Dictionary are allowed, but they have a fixed meaning, storage type, and units, so use them only for that data.

Object vs. Array Form

Object form — used when each row of the CSV file represents one vehicle record. The `col_defs` value is a single JSON object.

```
{
  "col_defs": {
    "veh_id":   { "func": "copy", "parms": [0], "type": "int" },
    "veh_time": { "func": "copy", "parms": [1] }
  }
}
```

Array form — used when each row of the CSV file encodes records for multiple vehicles in different column groups. The `col_defs` value is a JSON array where each element is a complete column definition object. The application processes each definition object against the same row and inserts a separate database record for each.

```
{
  "col_defs": [
    {
      "event_id":      { "func": "const", "parms": [1], "type": "int" },
      "veh_id":        { "func": "copy",  "parms": [0], "type": "int" },
      "veh_time":      { "func": "copy",  "parms": [2] },
      "lane_id":       { "func": "copy",  "parms": [5], "type": "int" },
      "veh_dist_trav": { "func": "copy",  "parms": [6] },
      "veh_speed":     { "func": "copy",  "parms": [3] }
    },
    {
      "event_id":      { "func": "const", "parms": [1], "type": "int" },
      "veh_id":        { "func": "copy",  "parms": [1], "type": "int" },
      "veh_time":      { "func": "copy",  "parms": [2] },
      "lane_id":       { "func": "copy",  "parms": [7], "type": "int" },
      "veh_dist_trav": { "func": "copy",  "parms": [8] },
      "veh_speed":     { "func": "copy",  "parms": [4] }
    }
  ]
}
```

In this example, columns 0 and 1 hold the IDs of two different vehicles, column 2 holds a shared timestamp, and the remaining columns hold each vehicle's lane, distance, and speed. Each CSV row produces two database records.

Array-form rules:

- **Keys must be unique within each definition set** (JSON object keys are inherently unique within one object).
- **Keys may — and typically must — repeat across definition sets.** Every definition set must include all required columns, so keys such as `veh_id` and `veh_time` naturally appear in every set.
- **A key that appears in multiple sets must declare the same `type` in every set.** Conflicting types across sets are a configuration error.

- **The database schema is the union of all keys across all sets**, in the order each key is first encountered. A record produced by one set stores null in any column defined only by other sets.

Required Columns

The following column keys must appear in every column definition set, and each must have a populating function (they may not be placeholders — see `func`). The application reports an error and refuses the configuration if any are missing.

Column Key	Description
`event_id`	Numeric identifier for the data collection event or run.
`veh_id`	Unique vehicle identifier within the event.
`veh_time`	Timestamp of the vehicle state record, in seconds relative to the event start.
`lane_id`	Application-normalized lane identifier used for plotting and analysis.
`veh_dist_trav`	Cumulative distance traveled along the vehicle trajectory.

Together, `event_id`, `veh_id`, and `veh_time` form the primary key of the events table: at most one record is stored per vehicle per timestamp per event, and duplicate records in the source data are silently ignored. These three columns may not be null: a record that gets no value for one of them is skipped, and the import log lists it by row number (see CSV File Requirements).

The application also relies on two properties of the data that a configuration can't enforce:

- **Every vehicle in an event travels in the same direction**, with `veh_dist_trav` increasing over time. Leader/follower identification, gap and headway measurement, proximity detection, and lane-change gaps all compare `veh_dist_trav` values within a lane.
- **Records are sampled on a common interval.** Gap detection, Gap Interpolation, Missing Data, Trajectory Stitching, and Advanced Statistics use each event's most common time step as the expected sampling interval.

Column Definition Items

Each column's value is a JSON object that may contain the following items.

`func`

The name of the import function used to populate the column. If omitted, the column is a **placeholder**: it is created in the database but no data is imported into it (all rows receive a null value). This is useful for reserving a column to be populated by a later processing step.

```
"my_placeholder": {}
```

When `func` is provided, it must be one of the function names listed in Supported Import Functions. Providing `parms` without `func` is a configuration error — omit both for a placeholder, or add the intended function. Required columns may not be placeholders.

`parms`

A JSON array of parameters passed to the import function. The meaning of each parameter depends on the function; for functions that read from the CSV row, source column indices are given as zero-based integers. See Supported Import Functions for each function's parameters.

Each function declares which of its parameters are source column indices. When the configuration is checked, those indices must be non-negative integers, and a function given too few parameters is reported. When the CSV file is checked, every declared index must exist in every row (see CSV File Requirements).

```
"veh_speed": { "func": "copy", "parms": [5] }
```

> **Note:** Earlier versions of the application used a `src_col` item to declare the source column. `src_col` is deprecated and now rejected with an error. Declare the source column as `parms[0]` of the appropriate function instead — for example, `{"src_col": 3}` becomes `{"func": "copy", "parms": [3]}`.

`type`

The data type used when storing the value in the database. Supported values are `"float"` (default), `"int"`, and `"text"`.

- `"float"` — continuous numeric measurements; stored as SQLite REAL.
- `"int"` — identifier and classification columns; stored as SQLite INTEGER. Floating-point strings such as `"3.0"` are parsed as a float first and then truncated to an integer.
- `"text"` — string values such as names and labels; stored as SQLite TEXT.

A value that can't be converted to the column's type is stored as null. For the `copy` function, an unconvertible `int` or `float` value is instead reported as an error when the CSV file is checked, and the import doesn't run (see CSV File Requirements).

```
"veh_id": { "func": "copy", "parms": [0], "type": "int" }
```

Forced types on recognized columns. Columns listed in the Data Dictionary have a fixed storage type that the application enforces regardless of the configured `type`. For example, `veh_id` is always stored as an integer and `event_name` is always stored as text. If a configuration declares a conflicting type for a recognized column — including by leaving `type` out, which means `"float"` — the application logs a warning and overrides it. Declaring the correct type explicitly keeps the import log clean and makes the configuration self-documenting.

`units`

The units of the source CSV data for this column. When provided, the application converts the value to the application's internal SI (meter-kilogram-second) units before storing it. If omitted, the value is stored as-is.

```
"veh_speed": { "func": "copy", "parms": [5], "units": "mph" }
```

How `units` is treated depends on the column:

Column	Effect of `units`
A Data Dictionary column with a unit in the **Units** column of the Data Dictionary	The value is converted to that unit. A unit string that isn't in Supported Unit Conversions, or one for a different quantity (for example `"mph"` on `veh_dist_trav`), is a configuration error.
A unitless Data Dictionary column (identifiers, codes, the geographic coordinates `x_map_origin`, `y_map_origin`, `veh_lat`, and `veh_lon`, and `event_name`)	Don't set `units`. Any unit string is a configuration error.
A user-defined column	No conversion. `units` has no effect, and a warning is logged.

Units are applied after the import function has produced the value. The `utm_lat` and `utm_lon` functions read their source columns directly, so their inputs must already be in decimal degrees and meters (see `utm_lat`).

Supported Unit Conversions

The application stores all physical quantities in SI (MKS) units. When source data uses different units, specify the source unit string in the `units` key and the application converts it automatically.

Source Units (in CSV)	Application Units (stored in database)
`s`	`s` — seconds (identity)
`m`	`m` — meters (identity)
`ft`	`m` — meters
`m/s`	`m/s` — meters per second (identity)
`mph`	`m/s` — meters per second
`kph`	`m/s` — meters per second
`m/s^2`	`m/s^2` — meters per second squared (identity)
`ft/s^2`	`m/s^2` — meters per second squared
`m/s^3`	`m/s^3` — meters per second cubed (identity)
`ft/s^3`	`m/s^3` — meters per second cubed

Unit strings are case-sensitive and must match exactly (for example, `km/h` and `ms` are not recognized).

Supported Import Functions

The application provides the following built-in functions for use in the `func` key. Each function receives the CSV row and the column's definition, and returns the value to store; the value is then converted to the column's `type` and, where applicable, its `units`.

`copy`

Copies a value from a CSV column into the database column, converted per the column's `type`. If the source cell is empty (or the row is shorter than the column index), the default value is stored if one is given; otherwise null is stored.

Parameters:

Position	Type	Required	Description
0	integer	yes	Zero-based index of the source CSV column.
1	any	no	Default value to store when the source cell is empty. Converted per `type` like any other value.

`copy` is the only function whose values are checked before the import: for an `int` or `float` column, a non-empty cell that isn't a number is reported as an error with its row number when the CSV file is checked.

Example — copy column index 5, no default:

```
"veh_speed": { "func": "copy", "parms": [5], "units": "mph" }
```

Example — copy column index 7, storing 0 when the cell is empty:

```
"sensor_flag": { "func": "copy", "parms": [7, 0], "type": "int" }
```

To store the same constant for every row regardless of the CSV content, use `const` instead.

`const`

Stores a constant value for every row. Reads nothing from the CSV file. The value is converted per the column's `type`; a value that can't be converted, or a missing parameter, stores null.

Parameters:

Position	Type	Required	Description
0	any	yes	The constant value stored for every row.

Example — mark every vehicle in the dataset as automated:

```
"veh_automation": { "func": "const", "parms": [1], "type": "int" }
```

`group`

Generates a sequential integer index for each distinct combination of values in one or more CSV columns. The first combination encountered in the file is assigned 1, the next new combination 2, and so on. This is how an `event_id` is usually derived when events are distinguished by a combination of fields in the CSV data.

Parameters:

Position	Type	Required	Description
0–n	integer	at least one	Zero-based indices of the CSV columns whose values form the group key.

Details:

- The key is the non-empty values of the listed columns, joined with single spaces. Empty cells are left out of the key, so rows that differ only in which column is empty can share a key (for example, `A`,(empty),`B` and `A`,`B`,(empty) both give `A B`).
- If every listed cell is empty, the value is null. For `event_id`, that means the row is skipped, since `event_id` may not be null.
- Numbering depends on the order of rows in the file, and starts again at 1 for every import. The same data in a different row order can produce different event numbers.
- Values are compared as text, so `1` and `1.0` are different keys.

Example — generate an event ID from the combination of values in columns 37, 38, 41, 42, and 43:

```
"event_id": { "func": "group", "parms": [37, 38, 41, 42, 43], "type": "int" }
```

`event_name`

Builds a human-readable event name by joining the values of one or more CSV columns, with an optional text prefix. The name is shown in the event selector, in the plot title, and in other places where events are listed.

Parameters:

Position	Type	Required	Description
0–n-1	integer	at least one	Zero-based indices of the CSV columns whose values form the name.
last	string	no	Optional text prefix. Recognized by being a string rather than a number.

Details:

- The non-empty values of the listed columns are joined with single spaces, and the prefix is put in front with no space added — include a trailing space in the prefix if you want one, as in `"run index: "`.
- If every listed cell is empty, the value is null and the application shows "Event N" for that event.
- Each event shows one name, so the listed columns should have the same values for every row of an event. Usually they are the same columns, or a subset of the columns, given to `group` for `event_id`.
- Store the result as `"type": "text"` (forced for `event_name`).

Example — build the event name from columns 42 and 43 with a "run index:" prefix:

```
"event_name": { "func": "event_name", "parms": [42, 43, "run index: "], "type": "text" }
```

`utm_lat`

Computes a vehicle's geographic latitude from a map origin and the vehicle's position relative to it.

The function converts the map origin's latitude and longitude to UTM coordinates (in the origin's UTM zone), adds the vehicle's east and north offsets to the origin's easting and northing, and converts the result back to latitude and longitude.

Parameters:

Position	Type	Required	Description
0	integer	yes	CSV column index of the map origin X coordinate — its **longitude**, in decimal degrees.
1	integer	yes	CSV column index of the map origin Y coordinate — its **latitude**, in decimal degrees.
2	integer	yes	CSV column index of the vehicle X position — its **easting offset** from the origin, in meters.
3	integer	yes	CSV column index of the vehicle Y position — its **northing offset** from the origin, in meters.

Details:

- The result is in decimal degrees. Don't set `units` on the column.
- The source values are read directly from the CSV row, before any unit conversion, so the offsets must be in meters and the origin in decimal degrees even if other columns use `units` to convert from feet.
- If any of the four cells is missing or isn't a number, the value is null.
- Offsets are added within the origin's UTM zone, which is accurate for the distances of a typical study site.

Example:

```
"veh_lat": { "func": "utm_lat", "parms": [25, 26, 5, 6] }
```

`utm_lon`

Computes a vehicle's geographic longitude. Uses the same parameters and calculation as `utm_lat` and returns the longitude, in decimal degrees.

Example:

```
"veh_lon": { "func": "utm_lon", "parms": [25, 26, 5, 6] }
```

`lane_tgsim_L1L2`

Computes a normalized lane identifier for TGSIM (Third Generation Simulation) datasets collected in the L1 and L2 configurations. The source lane codes include negative values for some lane types; this function returns the absolute value of the lane code, so that every lane identifier in the database is a non-negative integer.

Parameters:

Position	Type	Required	Description
0	integer	yes	Zero-based index of the source CSV column containing the raw TGSIM lane code.

A decimal code such as `-2.0` is truncated to an integer before its absolute value is taken. An empty cell or a non-numeric value gives null.

Use `copy` on the same source column to keep the raw code in `orig_lane_id`.

Example:

```
"lane_id":      { "func": "lane_tgsim_L1L2", "parms": [4], "type": "int" },
"orig_lane_id": { "func": "copy",            "parms": [4], "type": "int" }
```

`lane_tgsim_stationary`

Computes a normalized lane identifier for TGSIM datasets collected at a stationary sensor location. In this configuration, lane codes greater than 9 indicate a secondary roadway; this function subtracts 9 from those codes and leaves codes of 9 or below unchanged.

Parameters:

Position	Type	Required	Description
0	integer	yes	Zero-based index of the source CSV column containing the raw TGSIM stationary lane code.

As with `lane_tgsim_L1L2`, a decimal code is truncated to an integer first, and an empty cell or a non-numeric value gives null.

Example:

```
"lane_id": { "func": "lane_tgsim_stationary", "parms": [4], "type": "int" }
```

`tgsim_automated`

Determines whether a vehicle is automated from a CSV column containing the text `yes` or `no`. Returns 1 if the cell is `yes` (in any capitalization) and 0 for anything else — including `no`, an empty cell, or `yes` with surrounding spaces.

Parameters:

Position	Type	Required	Description
0	integer	yes	Zero-based index of the source CSV column containing the automation status string.

Example:

```
"veh_automation": { "func": "tgsim_automated", "parms": [10], "type": "int" }
```

CSV File Requirements

When an import starts, the CSV file is checked in full before any data is written. If the check finds any errors, they are listed in the import log and nothing is imported.

- **Format.** Comma-separated values in UTF-8 encoding, with standard CSV quoting.
- **Header row.** The first row is treated as a header, and skipped, if every non-empty cell in it contains at least one letter. Otherwise the first row is imported as data. A file containing only a header row is an error.
- **Row width.** Every row must have at least as many columns as the highest source column index any function uses, plus one. A shorter row is an error.
- **Values.** For `copy` into an `int` or `float` column, a non-empty cell that isn't a number is an error. Other functions return null for values they can't use.
- **Error limit.** The check stops after 100 errors, so fix the reported problems and run the import again to see any further ones.
- **Duplicates.** A record with the same `event_id`, `veh_id`, and `veh_time` as one already imported is skipped without a message.
- **Missing key values.** A record for which the configuration produces no `event_id`, `veh_id`, or `veh_time` — for example, from an empty cell — is skipped. The import log names the first 10 such rows and the column that had no value, and the final count of rows read gives the total skipped. The rest of the file is still imported.
- **Database errors.** Any other error while writing records stops the import, and the partial database is removed.

Rows are read in file order, so the order of rows determines event numbering when `event_id` uses `group`.

Data Dictionary

The following column names are recognized by the application. Some are required in every configuration (see Required Columns); the others are optional. A recognized column always has the storage type listed here.

Where the unit is m, s, m/s, m/s², or m/s³, the column accepts `units` for automatic conversion; columns in degrees, and columns with no unit, don't accept `units`.

Several application features use optional columns and are only available when those columns are present. Columns with nothing in the **Used by** column are stored and exported, but not used by the application itself.

Variable Name	Type	Units	Example	Description	Used by
`event_id`	int		1	**Required.** Identifier for a data collection event, run, or file segment. Used to distinguish records belonging to different trajectory datasets or collection periods.	Everything
`veh_id`	int		57	**Required.** Unique identifier assigned to each vehicle within an event.	Everything
`veh_time`	float	s	8.2	**Required.** Time at which the vehicle state is recorded, typically measured relative to the start of the event or run.	Everything
`lane_id`	int		5	**Required.** Application-normalized lane identifier. Number the lanes consecutively across the roadway: lanes whose IDs differ by 1 are treated as neighbors by the Rapid Lane Change anomaly, the Lane-Level Anomalies fix panel, and the stitching lane-skip check. Keep the source labels in `orig_lane_id`.	Everything
`veh_dist_trav`	float	m	97.47	**Required.** Cumulative distance traveled by the vehicle along its trajectory up to the recorded time.	Everything

Variable Name	Type	Units	Example	Description	Used by
`orig_lane_id`	int		5	Original lane identifier as it appears in the source dataset, preserved before any lane normalization.	Lane tooltips in the Time Space Plot Settings dialog and the plot's lane list
`veh_speed`	float	m/s	4.71	Instantaneous vehicle speed at the recorded time.	Show Speed coloring, plot tooltips, Time-Speed plot, Trajectory Statistics, Advanced Statistics speed statistics, Spline Fitting and Gap Interpolation (speed base and plots), Jerk Analysis (from speed), Internal Consistency comparison, Trajectory Stitching speeds
`veh_accel`	float	m/s^2	1.13	Instantaneous vehicle acceleration at the recorded time.	Plot tooltips, Time-Accel plot, Trajectory Statistics, Spline Fitting and Gap Interpolation (acceleration base and plots), Jerk Analysis (from acceleration), Internal Consistency comparison

Variable Name	Type	Units	Example	Description	Used by
veh_jerk	float	m/s³		Instantaneous rate of change of acceleration at the recorded time.	Jerk Analysis (direct), Spline Fitting (jerk base), Internal Consistency comparison
veh_length	float	m	5.39	Vehicle length.	Plot tooltips
veh_width	float	m	2.67	Vehicle width.	
preceding_veh_id	int			Identifier of the vehicle directly preceding the subject vehicle in the same lane, as given by the source data. (TRACES identifies leaders itself for Gap Headway mode and Advanced Statistics.)	
veh_spacing	float	m		Distance between the subject vehicle and its preceding vehicle at the recorded time, as given by the source data.	
veh_time_headway	float	s		Time headway between the subject vehicle and its preceding vehicle at the recorded time, as given by the source data.	
x_map_loc	float	m	378.57	Vehicle X position in a local map coordinate system. For utm_lat / utm_lon, the easting offset from the map origin.	
y_map_loc	float	m	211.69	Vehicle Y position in a local map coordinate system. For utm_lat / utm_lon, the northing offset from the map origin.	

Variable Name	Type	Units	Example	Description	Used by
x_frenet_loc	float	m		Vehicle position along the roadway reference path in a Frenet coordinate system.	
y_frenet_loc	float	m		Vehicle lateral offset from the roadway reference path in a Frenet coordinate system.	
x_map_origin	float	degrees		X coordinate (longitude) of the map origin or local reference point, in decimal degrees.	
y_map_origin	float	degrees		Y coordinate (latitude) of the map origin or local reference point, in decimal degrees.	
veh_lat	float	degrees		Vehicle latitude in decimal degrees, usually from utm_lat.	
veh_lon	float	degrees		Vehicle longitude in decimal degrees, usually from utm_lon.	
veh_automation	int		0	Indicator of the vehicle's automation or connectivity status. Typically 0 (not automated) or 1 (automated).	Plot tooltips
osm_way_id	int			Identifier of the associated OpenStreetMap way.	
osm_speed_limit	int			Speed limit associated with the roadway segment, from OpenStreetMap or another map source, stored as given.	
osm_traffic_control	int			Traffic control information for the roadway segment, such as stop or signal control.	

Variable Name	Type	Units	Example	Description	Used by
`event_name`	text		run index: 1	Human-readable name or label for the data collection event. When absent, the application generates a label of the form "Event 1".	Event selector, plot title, event lists

---

Columns Created by the Application

The application adds columns of its own to the events table as you use it. These names can't be used as keys in any `col_defs` configuration: the application rejects a configuration that uses one, with the error *"Column key '…' is reserved for internal computation and may not be configured."*

A configuration must not define them because a module that finds its column already present uses it as it is — a configured `imputed_status` column, for example, would hold nulls instead of the status codes, and every analysis would treat its rows as missing.

Row and Trajectory Status

Column	Created by	Contents
`imputed_status`	The first module that adds or changes rows: Spline Fitting, Gap Interpolation, or Lane-Level Anomalies	The provenance of each row, as a one-character code: blank for an original observation, `S` for a synthetic Spline Fitting row, `A` or `M` for a row filled automatically or manually by Gap Interpolation, `L` for a row whose lane was reassigned by Lane-Level Anomalies.
`traj_status`	Trajectory Stitching	The status of each trajectory: `K` kept, `R` removed (used in a stitch), `S` stitched (created by a stitch).

The *TRACES User Guide*, section 10 "Changing Data: Write Targets and Row Status", explains how each code is set and how the rest of the application treats it.

Analysis Results

Column	Created by	Contents
Any name beginning with `calc_` — currently `calc_position`, `calc_speed`, `calc_acceleration`, `calc_jerk`	Spline Fitting	Spline-derived kinematics
`veh_speed_cal`, `veh_accel_cal`, `veh_jerk_cal`	Internal Consistency	Speed, acceleration, and jerk calculated from the next lower kinematic

Column	Created by	Contents
`speed_error`, `accel_error`, `jerk_error`	Internal Consistency	Difference between each calculated value and the measured value

Other Reserved Names

Name	Reason
`is_spline_imputed`	Spline imputation flag used by databases created before `imputed_status`. TRACES copies it into `imputed_status` and no longer writes it.
`summary_json`	Per-event summary stored in the database's `ranges` table.
`time_diff`, `spacing_cal`	Reserved for future analysis results.

Tables Created by the Application

Besides the `events` table that the configuration defines, a TRACES database holds tables the application manages: `ranges` (written at the end of every import), `gaps`, `stitches`, `lane_anomalies`, `lane_corrections`, and `superseded_points` (the values of rows that a correction replaced). Their names don't conflict with column keys, but they shouldn't be created or edited by other tools.

Complete Example

The following example illustrates a complete configuration file for a CSV dataset of vehicle trajectories collected at an instrumented highway site. Distances and vehicle dimensions in the source file are in feet, speeds in mph, and accelerations in ft/s²; all are converted automatically. Empty acceleration cells default to 0.

```
{
  "file_ext": "csv",
  "col_defs": {
    "event_id":       { "func": "group",      "parms": [10, 11], "type": "int" },
    "event_name":     { "func": "event_name", "parms": [10, 11, "Site "], "type": "text" },
    "veh_id":         { "func": "copy",       "parms": [0], "type": "int" },
    "veh_time":       { "func": "copy",       "parms": [1] },
    "lane_id":        { "func": "copy",       "parms": [2], "type": "int" },
    "orig_lane_id":   { "func": "copy",       "parms": [2], "type": "int" },
    "veh_dist_trav":  { "func": "copy",       "parms": [3], "units": "ft" },
    "veh_speed":      { "func": "copy",       "parms": [4], "units": "mph" },
    "veh_accel":      { "func": "copy",       "parms": [5, 0], "units": "ft/s^2" },
    "veh_length":     { "func": "copy",       "parms": [6], "units": "ft" },
    "veh_width":      { "func": "copy",       "parms": [7], "units": "ft" },
    "site_code":      { "func": "copy",       "parms": [10], "type": "int" },
    "data_source":    { "func": "const",      "parms": ["site survey"], "type": "text" },
    "my_custom_tag":  {}
  }
}
```

In this example, `site_code` is a user-defined column that preserves the raw site identifier from the source data. `data_source` is a user-defined text column storing the same constant label for every row. `my_custom_`

`tag` is declared with an empty definition, creating a placeholder database column with null values for all rows — it would be populated by a subsequent processing step.

The bundled configurations in the `config` folder are further examples: `ohio-one-veh.json` and `ohio-two-veh.json` show `utm_lat` / `utm_lon`, and the `tgsim-*.json` files show the TGSIM lane and automation functions.

Adding Import Functions

New import functions, and new import processors for other file formats, are added in the application's code. How to implement them is covered in the *TRACES Developer Guide*, sections 7.3 "Adding an import function" and 7.4 "Adding a file-format importer". Once added, a new function is used in a configuration like any built-in one: by its name in `func`, with its parameters in `parms`.